

資料結構 Data Structure

HW01

組員1: 114310115 簡紘田

組員2: 114310123 盧敬恩

目錄

1. 使用的資料結構說明
 - Stack
 - Linked List
2. Infix-to-Postfix 演算法解釋
 - 轉換流程
 - 運算子優先權
3. Postfix 計算機實作
 - Postfix計算流程
 - Stack 在計算中的用途
4. 多位數與小數處理
5. 負數處理方式
6. 邏輯運算功能
 - AND
 - OR
 - XOR
 - NOT
7. 錯誤處理
8. 時間複雜度分析
9. 測試案例與執行結果
10. 遇到的問題與解決方式
11. 可能的改進與延伸功能
12. AI使用紀錄

1. 使用的資料結構說明

1.1 Stack

Stack 是一種「後進先出(LIFO)」的資料結構。

Stack 的主要有 push、pop、get、isEmpty的功能

在 Infix-to-Postfix 跟 evaluate Postfix 中有著大功用

1.2 Linked List

本專題中的 Stack 使用單向鏈結串列實作。

2. Infix-to-Postfix 演算法解釋

2.1 轉換流程

程式會逐字讀取輸入字串：

如果是數字直接輸出到 postfix

如果是左括號push 進 Stack

如果是右括號持續 pop, 直到遇到左括號

如果是運算子若 Stack 頂端運算子優先權較高或相同就先 pop

最後再將目前運算子 push 進 Stack

2.2 運算子優先權

反向(!)最優先, 優先權為5

次方(^)優先權為4

乘法(*)除法(/)餘數計算(%)優先權為3

加減法(+)(-)優先權為2

AND(&)OR(|)XOR(~)數字算完才做, 優先權為1

用int precedence(char op)來判斷運算順序。

3. Postfix 計算機實作

3.1 後綴式計算流程

讀取 postfix, 如果是數字就 push 進 Stack; 如果是運算子就 pop 出兩個數值進行計算, 因為是 stack 所以加數、乘數、除數、次方數等會較晚被 push 進, pop 時給值時也要注意(val1, val2)順序。

3.2 Stack 在計算中的用途

將運算的結果(result)塞回 stack 再拿出來當下一個val1或val2做運算

4. 多位數與小數處理

使用 isdigit() 辨認是否為數字, 並透過 while 持續讀取是否為數字或小數點, 最後用stod(numStr)轉換成double

5. 負數處理方式

(token == '-' &&(i == 0 || infix[i - 1] == '('))加入這兩個條件可以偵測減法與第一項的負數(第一項負數通常不會透過括號打包)

6. 邏輯運算功能

6.1 AND(&)

a && b

6.2 OR(|)

`a || b`

6.3 XOR(~)

`a != b`

只有一方為 true 時回傳 true。

6.4 NOT(!)

`!a`

進行布林反轉。

7. 錯誤處理

Stack Underflow

避免 pop 對空的Stack抽取

除以零

`if (val2 == 0)`

避免除以 0 導致商不存在

Modulo by zero

避免商不存在找不出餘數

Invalid operator

避免輸入沒有紀錄功能的運算子

8. 時間複雜度分析

```
bool isEmpty() const { //這裡問AI說要加上CONST 代表該函式只會讀取不會寫入 因為在string
infixToPostfix有用到isEmpty const Stack<char>&
    //編譯器在執行 s.isEmpty() 時, 內部的 this 指標型態是 const Stack<char>* 那麼一來便會更動到指標
    return (topNode == nullptr);
```

isEmpty :只會回傳(top是否為null)

時間複雜度:O(1)

```
void push(T val) {
    Node* newNode = new Node(val); //當push的時候 新建立一個節點 該點的位置就是newnode
    newNode->next = topNode; //top由原本的 (可能是null)的新節點的next
    topNode = newNode;
}
```

push: 將新內容push進stack,把top指標改指向該內容 (不管甚麼資料都是三步驟)

時間複雜度:O(1)

```
T pop() {
    if (isEmpty()) {
        throw runtime_error("Stack Underflow");
    }
    Node* temp = topNode; //temp指標指向當前的top
    T poppedData = temp->data; //變數popdata為堆疊表面的data
    topNode = topNode->next; //表層被pop了, top變成原本top的next
    delete temp;
    return poppedData;
}
```

POP 將TOP改為原本的top的next 回傳原本TOP並刪除

時間複雜度:O(1)

```
T get() const { //這裡是取得堆疊top資料但是不pop
    if (isEmpty()) {
        throw runtime_error("Stack is empty");
    }
    return topNode->data;
}
```

get:回傳top但是不修改堆疊時間複雜度:O(1)

```
int precedence(char op) { //在infix to postfix的時候 要先pop 優先權>=token的運算元
    if (op == '!') return 5; // 反向優先權最高
    if (op == '^') return 4; // 次方優先於其他四則運算子
    if (op == '*' || op == '/' || op == '%') return 3; // 先乘除後加減所以優先權大於加減
    if (op == '+' || op == '-') return 2;
    if (op == '&' || op == '~' || op == '|') return 1; // 邏輯運算最後才做

    return 0;
}
```

優先權計算:時間複雜度:O(1)

```

string infixToPostfix(const string& infix) {
    Stack<char> s;
    string postfix = "";

    for (size_t i = 0; i < infix.length(); i++) {//infix有多長postfix就有多長
        char token = infix[i];

        if (isspace(token)) continue;//isspace 為c++函式 如果空白就跳過

        if (isdigit(token) || token == '.' || (token == '-' && (i == 0 || infix[i - 1] == '(') && (isdigit(infix[i + 1]) || infix[i + 1] == '.'))) //token讀到 '.' 和數字時，負號則要位於開頭或是後面
            if (infix[i] == '-') {
                postfix += infix[i++];
            }

            while (i < infix.length() && (isdigit(infix[i]) || infix[i] == '.')) {
                postfix += infix[i++];
            }
            postfix += " ";//輸出完一組數值後 輸出空白
            i--; //此時因為while break i會多加一次 要-掉1
        }
        else if (token == '(') //這裡開始就是一般的將 ( push 然後
            s.push(token);
        }
        else if (token == ')') {
            while (!s.isEmpty() && s.get() != '(') //pop直到出現 ) 就是pop 到出現 ) 為止 因為誇號不輸出postfix 所以用get
                postfix += s.pop();
                postfix += " ";//這裡的空白是關鍵 用以區分數字和數字
            }
            if (!s.isEmpty()) s.pop();
        }
        else {
            while (!s.isEmpty() && precedence(s.get()) >= precedence(token)) //這裡是當token是運算元時 pop優先權的>=token的運算元
                postfix += s.pop();
                postfix += " ";//這裡的空白是關鍵 用以區分數字和數字
            }
            s.push(token);
        }
    }

    while (!s.isEmpty()) { //還有東西就全部pop
        postfix += s.pop();
        postfix += " "; // 這裡的空白是關鍵 用來區分數字和數字
    }

    if (!postfix.empty()) postfix.pop_back();

    return postfix;
}

```

INFIX=>POSTFIX:這裡使用了很多迴圈,但是沒有巢狀結構 所以時間複雜度= $O(n)$

```

bool logical(bool a, bool b, char op) { // 負責邏輯運算
    switch (op) {
        case '&':
            return a && b;
        case '|':
            return a || b;
        case '~':
            return a != b;
        default:
            throw runtime_error("Invalid logical operator");
    }
}

bool logical_NOT(bool a) { // 只做反向
    return !a;
}

```

反向和邏輯運算,都是O(1)

```

double evaluatePostfix(const string& postfix) { //將post fix 算出
    Stack<double> s;
    for (size_t i = 0; i < postfix.length(); i++) {
        if (isspace(postfix[i])) continue; // isspace 為c++函式 如果空白就跳過

        if (isdigit(postfix[i]) || postfix[i] == '.' || (postfix[i] == '-' &&
i + 1 < postfix.length() && (isdigit(postfix[i + 1]) || postfix[i + 1] ==
'.'))) { //postfix[i]讀到'.' 和數字時, 負號則要位於開頭或是(後面
            string numStr = "";
            if (postfix[i] == '-') {
                numStr += postfix[i++];
            }
            while (i < postfix.length() && (isdigit(postfix[i]) || postfix[i]
== '.')) {
                numStr += postfix[i++];
            }

            s.push(stod(numStr));

            i--;
        }
        else {
            if (postfix[i] == '!') { // 優先做反向
                double val = s.pop();
                bool result = logical_NOT(val != 0);
                cout << "!" << val << " = " << result << endl;
                s.push(result);
                continue;
            }
            double val2 = s.pop();

            double val1 = s.pop();

            cout << val1 << " " << postfix[i] << " " << val2 << " = ";
            switch (postfix[i]) { //計算與結果輸出

```



```

        case '+':
            s.push(val1 + val2);
            cout << val1 + val2 << endl;

            break;
        case '-':
            s.push(val1 - val2);
            cout << val1 - val2 << endl;

            break;
        case '*':
            s.push(val1 * val2);
            cout << val1 * val2 << endl;

            break;
        case '/':
            if (val2 == 0) { // 除以0會使商不存在, 所以視為錯誤輸出
                throw runtime_error("Division by zero");
            }
            s.push(val1 / val2);
            cout << val1 / val2 << endl;

            break;
        case '^':
            s.push(pow(val1, val2));
            cout << pow(val1, val2) << endl;

            break;
        case '%':
            if (val2 == 0) { // 除以0會使商不存在, 找不到餘數, 所以視為錯誤輸
                throw runtime_error("Modulo by zero");
            }
            s.push(fmod(val1, val2));
            cout << fmod(val1, val2) << endl;

            break;
        case '&':
        case '|':
        case '~':
        {
            bool result = logical(val1 != 0, val2 != 0, postfix[i]);
            // 丟進 logical 運算
            s.push(result);
            cout << result << endl;
            break;
        }
        default:
            throw runtime_error("Invalid operator");
    }
}

return s.pop(); // 跑完for最後pop答案

```

```
}
```

這裡是輸出postfix的結果 雖然for迴圈裡面包著while,但是不是 $O(N^2)$ 因為WHIHE迴圈由當前的i開始做起直到i到達了string的長度, 所以時間複雜度僅有 $O(n)$

9. 測試案例與執行結果

為了檢測所有功能所以選擇了 $-2*(-9.5 + 2^3)/0.5*4\%7$ 這條式子
首先是開頭與'('後的'-'將視為負號而不是減法才不會導致錯誤
再來是先做次方才作加法
最後是/、*與%的功能

```
=====
  中綴式轉後綴式並求值計算機 (輸入 'exit' 離開)
=====

請輸入算式 (例如 -2*(-9.5 + 2 ^ 3)/0.5 * 4 % 7):
> -2*(-9.5 + 2 ^ 3)/0.5 * 4 % 7
後綴式 (Postfix) : -2 -9.5 2 3 ^ + * 0.5 / 4 * 7 %
2 ^ 3 = 8
-9.5 + 8 = -1.5
-2 * -1.5 = 3
3 / 0.5 = 6
6 * 4 = 24
24 % 7 = 3
計算結果 (Result) : 3
-----

請輸入算式 (例如 -2*(-9.5 + 2 ^ 3)/0.5 * 4 % 7):
> |
```

再來是邏輯運算

```
=====
  中綴式轉後綴式並求值計算機 (輸入 'exit' 離開)
=====

請輸入算式 (例如 -2*(-9.5 + 2 ^ 3)/0.5 * 4 % 7):
> 1 & 0 | !0 ~0
後綴式 (Postfix) : 1 0 & 0 ! | 0 ~
1 & 0 = 0
!0 = 1
0 | 1 = 1
1 ~ 0 = 1
計算結果 (Result) : 1
-----

請輸入算式 (例如 -2*(-9.5 + 2 ^ 3)/0.5 * 4 % 7):
> |
```

與作業要求中的例題

```
=====
  中綴式轉後綴式並求值計算機 (輸入 'exit' 離開)
=====

請輸入算式 (例如 -2*(-9.5 + 2 ^ 3)/0.5 * 4 % 7):
> 3+4-2
後綴式 (Postfix) : 3 4 + 2 -
3 + 4 = 7
7 - 2 = 5
計算結果 (Result) : 5
-----

請輸入算式 (例如 -2*(-9.5 + 2 ^ 3)/0.5 * 4 % 7):
> 2*3+4
後綴式 (Postfix) : 2 3 * 4 +
2 * 3 = 6
6 + 4 = 10
計算結果 (Result) : 10
-----
```

```
請輸入算式 (例如 -2*(-9.5 + 2 ^ 3)/0.5 * 4 % 7):
> 2*(3+4)
後綴式 (Postfix) : 2 3 4 + *
3 + 4 = 7
2 * 7 = 14
計算結果 (Result) : 14
-----

請輸入算式 (例如 -2*(-9.5 + 2 ^ 3)/0.5 * 4 % 7):
> 10/2+3
後綴式 (Postfix) : 10 2 / 3 +
10 / 2 = 5
5 + 3 = 8
計算結果 (Result) : 8
-----

請輸入算式 (例如 -2*(-9.5 + 2 ^ 3)/0.5 * 4 % 7):
> 2^3+1
後綴式 (Postfix) : 2 3 ^ 1 +
2 ^ 3 = 8
8 + 1 = 9
計算結果 (Result) : 9
-----
```

```

請輸入算式 (例如 -2*(-9.5 + 2 ^ 3)/0.5 * 4 % 7):
> 25%7+10
後綴式 (Postfix) : 25 7 % 10 +
25 % 7 = 4
4 + 10 = 14
計算結果 (Result) : 14
-----

請輸入算式 (例如 -2*(-9.5 + 2 ^ 3)/0.5 * 4 % 7):
> (2+3)*(4-1)
後綴式 (Postfix) : 2 3 + 4 1 - *
2 + 3 = 5
4 - 1 = 3
5 * 3 = 15
計算結果 (Result) : 15
-----

請輸入算式 (例如 -2*(-9.5 + 2 ^ 3)/0.5 * 4 % 7):
> 2.5+3.5*2
後綴式 (Postfix) : 2.5 3.5 2 * +
3.5 * 2 = 7
2.5 + 7 = 9.5
計算結果 (Result) : 9.5

```

10. 遇到的問題與解決方式

問題一：無法處理負數

開頭的負數如-2*程式會把 - 當成減法。

解決方式

加入($i == 0 \parallel \text{infix}[i - 1] == '('$)判斷是否為負號。

問題三：無法區分數字

Postfix沒有空格(1234+)會不知道是1 2 34還是12 3 4

解決方式

在每組 token 後加入空白。

問題四：無法把小數點視為一個數

當使用者輸入3.3時,程式無法判別小數點

解決方式

用c++ 的template T 當使用者輸入2.6的時候 c++會把t設定為2.6
PUSH T 的時候就能直接對T(2.6)操作

11. 可能的改進與延伸功能

11.1 比較運算子

>、<、=...

11.2 三角函數和其他常用數學常數等

sin()、cos()、 π 、e...

11.3 GUI 圖形介面

目前為 Console, 可改成視窗版。

11.4 歷史紀錄功能

儲存過去算式與結果。

11.5 變數功能

$x = 5$, $y = 3$, $x + y = 8$

12. AI使用紀錄

template T:處理小數點問題

修改程式在in=>postfix一個數值結束後加入空白

如果你一開始完全不知道 `template`（範型）是什麼，我們可以把它想像成「一個還沒決定尺寸和形狀的 3D 列印模型（藍圖）」。

在這個題目中，我們要處理兩種完全不同的東西：

- 1 符號（如 `+`，`-`，`*`，`/`）：在電腦裡叫作 `char`（字元）。
- 2 帶小數點的數字（如 `21.4`，`3.0`）：在電腦裡叫作 `double`（倍精度浮點數）。

如果你不知道 `template`，你可能會想：「糟糕，那我是不是要寫一個專門裝符號的 `Stack`，還要再寫一個專門裝小數點數字的 `Stack`？」

不用！這就是 `template` 施展魔法的地方。

第一步：用 `T` 來當作「百變怪」的代號

在你的程式碼中，有一行長這樣：

C++

```
template <typename T>
class Stack {
    struct Node {
        T data; // 📌 關鍵就在這個 T ！
```

這裡的 `T` 是一個虛擬的代號。它現在什麼都不是，它只是一個空下來的插槽。

這行程式碼的意思是告訴電腦：「我設計了一個堆疊的排隊機制（指標怎麼接、怎麼 `Push`、怎麼 `Pop`），但裡面到底要裝什麼型態的貨物？我先不決定，用 `T` 來代替！」

第二步：在主程式中，把 `T` 自由插上不同的晶片

因為你把型態留空（變成了 `T`），當你在題目中實際要使用這個 `Stack`時，你就可以像插卡帶一樣，自由地決定這次的 `T` 要變成什麼：

1. 當你想放入「符號」時：

你在主程式寫下這行：

C++

```
Stack<char> opStack;
```

和計算postfix的時候利用是否有空白來判定數字是誰

我有一串postfix計算結果 電腦如何知道數字和數字的格和小數點



電腦之所以能分得清「這是 21.4」還是「這是 2 和 1.4」，最關鍵的幕後功臣就是我們在轉換時故意塞進去的「空格」。

在電腦的世界裡，後綴表達式（Postfix）不是一整串黏在一起的文字，而是被看作一連串被空格切開的 **Token**（標記/單元）。

電腦在讀取這串字串時，通常會採取以下兩種方法的其中一種：

方法一：指標走位法（也就是你 code 裡寫的方法）

電腦會用一個指標 `i` 從左到右像毛毛蟲一樣一格格爬行。這時候它會啟動我們剛才討論的那個 `while` 迴圈：

- 1 遇到數字或小數點：迴圈啟動，把字元一個個吸過來黏在 `numStr` 屁股（例如先吸 `'2'`、再吸 `'1'`、再吸 `'.'`、再吸 `'4'`）。
- 2 遇到空格：`while` 迴圈的條件（必須是數字或小數點）瞬間破功（`False`），迴圈立刻踩煞車停下來！
- 3 打包推入 `Stack`：電腦知道這個數字抓完了（變成 `"21.4"`），於是轉成 `double` 推入 `Stack`。
- 4 跳過空格繼續走：下一次指標跳過空格，走到下一個位置。

因為每個數字後面都撞到一面叫「空格」的牆壁，所以電腦絕對不會把前後兩個數字混在一起。

作業翻譯，確保沒有會錯意

Assignment 01 (HW 01)
Calculator Implementation

Overview

In this group project (2-3 students per group), you will implement a calculator application that evaluates mathematical expressions. This project builds upon the concepts of linked lists and stacks we have covered in class, particularly the infix-to-postfix conversion exercise from Lab.

顯示更多

作業 01 (HW 01)

計算機實作 (Calculator Implementation)

概述 (Overview)

在這個小組專題中（每組 2 ~ 3 人），你們將實作一個可以計算數學表達式的計算機程式。這個專題會建立在課堂中學過的 linked list（鏈結串列）與 stack（堆疊）概念上，特別是 Lab 中做過的 infix-to-postfix（中序轉後序）轉換。

評分標準 (Evaluation Criteria)

核心功能正確性 (60%)

擴充功能與額外特色 (10%)

文件與分析報告 (30%)

額外加分 (最高 15%)

↓

+ 想問什麼都可以

🔊

🔇

ChatGPT 可能會出錯，請查核重要資訊。

產生報告目錄讓報告製作快速有個方向，再根據題目要求留下較為重要的部份

Assignment 01 (HW 01) Calculator Implementation Overview In this group project (2-3 students per group), you will implement a calculator application that evaluates mathematical expressions. This project builds upon the concepts of linked lists and stacks we have covered in class, particularly the infix-to-postfix conversion exercise from Lab. Evaluation Criteria ● Core implementation correctness (60%) ● Extensions and additional features (10%) ● Documentation and analysis (30%) ● Bonus (Up to 15%) Project Description: You will create a calculator that can evaluate arithmetic expressions with the following features: 1. Support for basic arithmetic operations (+, -, *, /) 2. Support for additional operations

顯示更多

你這份專題其實很適合做成「正式專題報告格式」，下面這份目錄是依照老師要求、評分項目，以及你們計算機專題內容去設計的，直接可以放進報告。

目錄 (Table of Contents)

1. 專題簡介 (Introduction)

1.1 專題背景

1.2 專題目的

1.3 專題功能介紹

2. 系統設計 (System Design)

2.1 系統架構

2.2 Stack 資料結構設計

2.3 Linked List 實作方式

2.4 運算子優先順序設計

3. 演算法說明 (Algorithm Description)

3.1 Infix Expression 與 Postfix Expression

↓

請AI根據我的程式碼跑過程，較容易觀察有無動作錯誤的地方

$$(-3.5 + 2^3) * 4 \% 7$$

能給我這個輸入的stack完整動作流程嗎

輸入中綴式:

$$(-3.5 + 2^3) * 4 \% 7$$


一、中綴式轉後綴式流程

初始狀態

Operator Stack: 空

Postfix: 空

讀到 (

Operator Push (

Operator Stack: (

Postfix:



讀到 -3.5

直接輸出到 postfix

Operator Stack: (

